THE AUSTRALIAN NATIONAL UNIVERSITY

Second Semester Final Examination – November 8th, 2021

COMP3320/6464/HONS
HIGH PERFORMANCE SCIENTIFIC COMPUTATION

- Time allowed: 3 hours and 30 minutes (including submission time to Wattle)
- Permitted materials: open-book exam
- Exam questions total 100 marks
- All students answer all questions
- Clarity and conciseness in answers will be highly valued
- Marks may be lost for supplying irrelevant information

1. Short Questions [20 marks]

a) Each of the following statements are either true or false. Which are they? Give a few sentences to explain each of your answers.

i. The relative error of a floating-point addition operation is approximately the sum of the relative errors of the two operands.

[2 marks]

ii. In a floating-point number system, the underflow limit is the smallest positive machine number that perturbs the number 1 when added to it.

[2 marks]

b) The roots of a quadratic equation $ax^2 + bx + c = 0$ are given by the following formula

$$x = \frac{-b \mp \sqrt{b^2 - 4ac}}{2a}$$

i) Explain the main errors associated with the evaluation of the above formula when using floating-point arithmetic.

[2 marks]

ii) Provide a strategy to decrease these floating-point errors which does not include using higher floating-point precision.

[2 marks]

c) For the following Pthread program

```
#define N 2
double *ptr1, *ptr2;
double dot = 0;
sem_t mutex;

void *thread(void *vargp){
  int *myIDp = (int*)vargp;
  for(int j = 0; j < 2; j++){
    int offset = *myIDp*2+j;
    sem_wait(&mutex);
    dot += ptr1[offset]*ptr2[offset];
    sem_post(&mutex);
  }
  return NULL;
}

int main(){
  pthread_t TID[N];
  double a[4] = {0.2,0.5,1.0,0.7};
  double b[4] = {0.3,0.5,2.3,9.0};
  ptr1 = a;
  ptr2 = b;
  sem_init(&mutex, 0, 1);
  for(int i = 0; i < N; i++)
    pthread_create(&TID[i], NULL, thread, (void*)&i);
  for(int i = 0; i < N; i++)
    pthread_join(TID[i], NULL);
  printf("dot = %f\n", dot);
  exit(0);
  return 0;
}
```

discuss how each of the variables `a`, `b`, `i`, `myIDp`, `dot`, `ptr1` and `ptr2` is mapped to memory, how many instances of each variable there are, and which ones are shared variables according to the Pthread memory model.

[6 marks]

d) In the following, `x`, `y`, and `z` are double precision floating point arrays, while `a`, `b`, and `c` are double precision floating point scalar variables.

i) `for (i=0; i<n; i++) a = a + b*x[i]*y[i]*z[i];`

ii) `for (i=0; i<n; i++) x[i] = (a*y[i]+b*y[i])*c;`

Would one of these loops significantly outperform the other on a contemporary x86 processor such as the ones on the Gadi system used in this course? Provide a detailed explanation for your answer assuming that the cache uses a write-allocate, write-back policy.

[4 marks]

e) Outline the main reasons for the exponential increase in FLOP performance of serial processors during the uniprocessor golden era.

[2 marks]

2. Shared Memory Programming with OpenMP [20 marks]

Consider three distinct matrices A, B and C. The A, B and C matrices are all $n \times n$ and symmetric, meaning that $A = A^T$, $B = B^T$ and $C = C^T$, where A^T is the matrix transpose of A. The following code performs the symmetric rank 2 update $C = A \times B^T + A^T \times B$ operation using a single thread.

```
for(i = 0; i < n; ++i){
  for(j = 0; j < n; ++j){
    C[i][j] = 0.0;
  }
}

for(i = 0; i < n; ++i){
  for(j = 0; j < n; ++j){
    for(k = 0; k < n; ++k){
      C[i][j] += A[i][k]*B[j][k]+A[k][i]*B[k][j];
    }
  }
}
```

Two attempts to develop an OpenMP parallel implementation are made:

Parallel version #1

```
#pragma omp parallel
{
#pragma omp for nowait
for(i = 0; i < n; ++i){
  for(j = 0; j <= i; ++j){
    C[i][j] = 0.0;
  }
}

#pragma omp for
for(i = 0; i < n; ++i){
  for(j = 0; j < n; ++j){
    for(k = 0; k < n; ++k){
      C[i][j] += A[i][k]*B[j][k]+A[k][i]*B[k][j];
    }
  }
}
}
```

Parallel version #2

```
#pragma omp parallel
{
#pragma omp for
for(i = 0; i < n; ++i){
  for(j = 0; j <= i; ++j){
    C[i][j] = 0.0;
  }
}

#pragma omp for
for(i = 0; i < n; ++i){
  for(k = 0; k < n; ++k){
    for(j = 0; j < n; ++j){
      C[i][j] += A[i][k]*B[j][k]+A[k][i]*B[k][j];
    }
  }
}
}
```

You test both implementations using multiple threads on a variety of different computer systems. You find that under some circumstances both versions give incorrect results.

a) In each case what modifications to the OpenMP directives would you make to ensure that both parallel implementations are always correct. You should make no modifications to the basic loop structure, but you are free to move, rewrite or add new OpenMP directives.

[4 marks]

b) How does the number of FLOPs in calculation scale with the parameter `n`? Provide an analytic expression in terms of `n` and not just its big O complexity argument.

[4 marks]

c) Rewrite the routine so that it runs correctly in parallel using OpenMP and with high efficiency. You are free to change the code as you like, adding additional local variables if required, as long as the code yields correct results. Explain briefly why you performed each optimization.

[6 marks]

d) Write a thread-parallel and vectorized version of the code using OpenMP directives. Discuss the motivation of your design choices in writing your code.

[6 marks]

3. High Performance Computer Architecture [20 marks]

a) You have a load/store computer that can issue one instruction per cycle, but cannot issue the next instruction until the previous one is complete. The computer runs the following instruction mix with that have the following times to complete:

OPERATION	FREQUENCY	# CYCLES TO COMPLETE
FADD	22%	2
FMUL	15%	2
LOAD	17.5%	4
STORE	22.5%	4
BRANCH	23%	6

where FADD and FMUL are floating point additions and multiplications, respectively.

(i) Compute the overall number of cycles per instruction (CPI). Show exactly how you derive your result.

[4 marks]

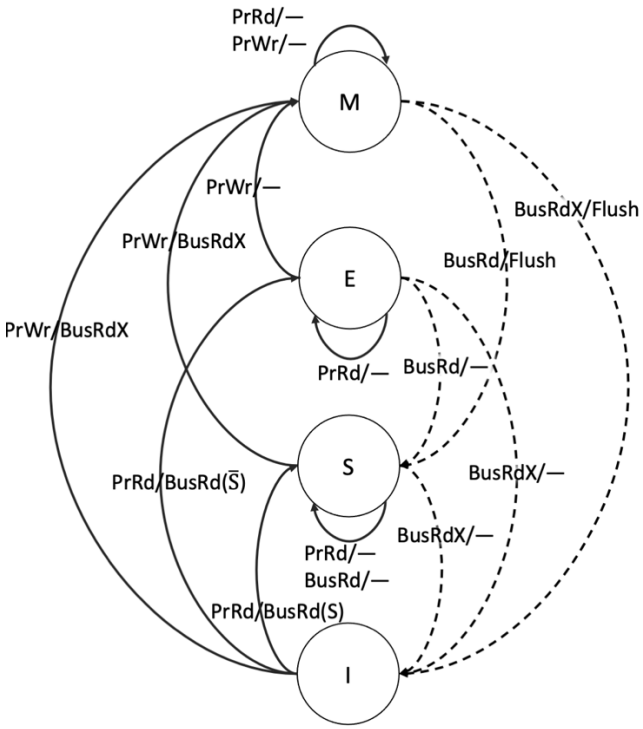
(ii) You observe that 10% of the FADD operations are paired with one FMUL, so you propose to replace these floating-point operations with a single new instruction, a floating-point fused multiply-add FFMA. The new instruction takes 2 clock cycles to complete, but in the revised architecture branches now take 7 cycles to complete. What is the new CPI value? Show exactly how you derive your new CPI value.

[4 marks]

(iii) If the ratio of clock speeds between the initial processor and the one proposed in (ii) is Clock old/Clock new = 1.3, which processor has the faster execution time and by what percent? Show exactly how you derive your result.

[4 marks]

b) Your machine implements (in hardware) the MESI cache coherence protocol shown in the following state diagram



The table below describes the behaviour of a program running in parallel using two processors P1 and P2 on your machine. C1 and C2 are cache lines associated with processors P1 and P2. The notation &X indicates the address of variable X, while mem[&X] is the content of the memory cell (*i.e.* the value of X) at address &X. Note that the "bus transactions" are initiated as a consequence of the processor action and they correspond to the solid lines in the protocol (processor action/bus transaction), while the bus snooper transactions correspond to the dashed lines.

Following the MESI protocol, fill the entries in the table based on the listed processor actions.

processor action	P1 cache miss/hit, X value	P2 cache miss/hit, X value	Bus transaction (processor action initiated)	Bus snooper transaction	C1 state	C2 state	mem[&X]
No-Op	N/A	N/A	–	–	I	I	0
P1Rd X	miss, 0	N/A	BusRd(S̄)	–			
P1Wr X = 4							
P2Rd X							
P1Wr X = X*2							
P2Wr X = X-3							

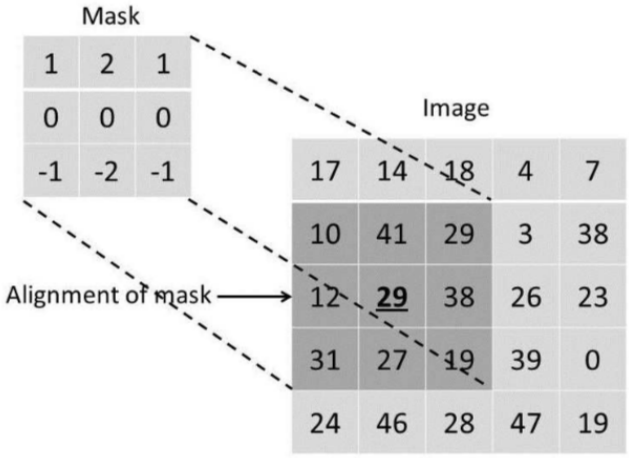
[8 marks]

4. Application of High-Performance Computing [20 marks]

One of the simplest operations in digital image processing is correlation. This operation computes the two-dimensional cross-correlation of two input matrices. The first matrix is an image where each element is a pixel. The second matrix, which is normally much smaller than the image, is a mask with values that correspond to weights. The correlation operation involves overlaying the mask on top of each pixel in the image and using it to compute a new pixel value that is a weighted sum of the surrounding "covered" pixels.
Consider an input image (I) and mask (M) defined as follows:

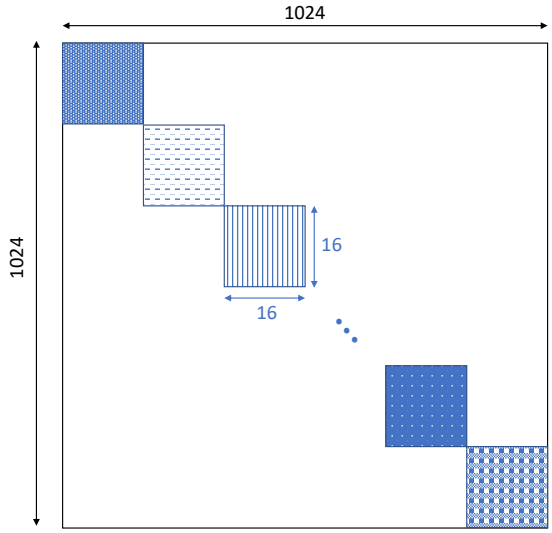
$$\mathrm{I} = \begin{bmatrix} 17 & 14 & 18 & 4 & 7 \\ 10 & 41 & 29 & 3 & 38 \\ 12 & 29 & 38 & 26 & 23 \\ 31 & 27 & 19 & 39 & 0 \\ 24 & 46 & 28 & 47 & 19 \end{bmatrix} \qquad \mathrm{M} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

When the mask is positioned over pixel I(3,2)



the new pixel value of $\mathrm{I}_{\mathrm{new}}(3,2)$ after the correlation is: 1*10 + 2*41 + 1*29 + 0*12 + 0*29 + 0*38 + (-1)*31 + (-2)*27 + (-1)*19 = 17.

Suppose that we use the same mask as above but for an image of 1,024×1,024 pixels. The image is stored as a 1,024×1,024 block-diagonal matrix, where each non-zero block is of dimension 16×16, as shown in the picture below.



The correlation operation (without processing the image boundary pixels) is performed using the following code (assume `Inew` is initialized to zero):

```
for(i = 1; i < 1023; ++i)
  for(j = 1; j < 1023; ++j)
    for(x = 0; x < 3; ++x)
      for(y = 0; y < 3; ++y){
        Inew[j][i] += I[j+x-1][i+y-1]*M[x][y];
}
```

Both the image and the mask are of type double (8 byte). The image data and the mask data are stored in row-major format in contiguous memory locations.

(i) The above code is executed on a computer with a 32KB level 1 (L1) cache that has a 64-byte line size. The cache uses write-back, write-allocate policy. Comment on the L1 cache usage, specifically discussing the L1 cache hit rate for both reads and writes of the above code. In your discussion detail any assumptions you make. **[5 marks]**

(ii) Rewrite the above code to make it more efficient by
   a. Making it more cache friendly. **[5 marks]**
   b. Reducing the computational scaling of the algorithm. **[5 marks]**

(iii) Consider that you have an 8-core CPU that supports SSE2 vector extensions. Write a parallel and vectorized version of the code and discuss the potential performance improvements over its serial version. **[5 marks]**

5. Performance Modelling [20 marks]

The weak speedup of an OpenMP parallel code running on *N* threads, each pinned to a distinct CPU core, can be modelled as

$$S_w(N) = \frac{f + (1-f)\,N^{\alpha}}{f + (1-f)N^{\alpha-1} + \kappa\, N^{\gamma}}$$

where *f* is a constant associated with the serial fraction of the application code, and κ and γ are positive constants.

a) Discuss the meaning of the γ term and provide two examples of different γ values to support your statements.

[4 marks]

b) Assuming $f = 0.1$ and $\kappa = 0$

(i) What is the maximum strong speedup S_s^{max} achievable by the code and for what is number of threads N_s^{max} for which S_s^{max} is achieved?

(ii) What is the strong parallel efficiency of the code when using N_s^{max} threads?

[6 marks]

c) For another application code we observe that $f = 0.07$, $\kappa = 0.05$ and $\gamma = 1$. For a fixed problem size, what is the number of threads that minimizes execution time?

[4 marks]

d) By performing a weak scaling analysis for your application code with $\alpha = 2$ and $s = 0.08$ your colleague obtained the following weak speedup data

S_w	N (number of threads)
1.68421	2
2.34711	3
2.93750	4
3.46667	5
3.94366	6
4.37584	7
4.76923	8
5.12883	9
5.45882	10

calculate the κ and γ parameters and discuss their meaning in your application code.

[6 marks]